ASSIGNMENT - 1

APACHE HIVE TOOL
FOR BIG DATA ANALYSIS

Student Name	:	Harshitha R
USN	:	4GW23CI015
Semester	:	7th SEM
Subject	:	MLOps
Assignment	:	Assignment - 1
Tool Selected	:	Apache Hive

1. SCENARIO

A retail company generates a large amount of sales transaction data from its stores and online sales platform. The dataset contains information such as transaction ID, product name, product category, quantity sold, price, and sale date.

As the volume of sales data increases, analysing the data using traditional methods becomes difficult and time-consuming. The company needs a system that can store and process large datasets and perform analytical queries efficiently.

Apache Hive is selected as the Big Data tool because it provides an SQL-like query language called HiveQL and can be used to analyse large datasets in the Hadoop ecosystem.

In this assignment, a small retail sales dataset is used to demonstrate how Apache Hive can be used to create a database and table, load data, execute queries, perform aggregation, calculate revenue, and identify top-selling products.

2. PROBLEM STATEMENT

Design and implement a simple retail sales analysis system using **Apache Hive**. The system should:

1. Create a Hive database.
2. Create a sales table.
3. Load a CSV sales dataset into the Hive table.
4. Display and verify the stored records.
5. Perform category-wise sales analysis.
6. Calculate total quantity sold.
7. Calculate category-wise revenue.
8. Identify top-selling products.
9. Display the final analytical results.

3. OBJECTIVES

1. To understand the basic working of Apache Hive.
2. To create and manage databases and tables using HiveQL.
3. To load structured CSV data into a Hive table.
4. To perform SQL-like queries on data using HiveQL.
5. To perform aggregation and grouping operations.
6. To calculate sales quantity and revenue.
7. To identify top-selling products.
8. To understand the application of Hive in large-scale data analysis.

4. SOFTWARE / TOOLS REQUIRED

Component	Details
Big Data Tool	Apache Hive
Query Language	HiveQL
Platform	Hadoop Ecosystem
Operating System	Linux
Input Dataset	CSV
Interface	Hive Shell / Beeline

5. DATASET

The following small retail dataset is used throughout this assignment. It is stored as a comma-separated file **sales.csv** at the path `/home/student/sales.csv`.

Transaction_ID	Product	Category	Quantity	Price	Sale_Date
101	Laptop	Electronics	2	55000	2026-09-01
102	Mouse	Accessories	5	1200	2026-09-02
103	Keyboard	Accessories	3	2500	2026-09-02
104	Monitor	Electronics	2	18000	2026-09-03
105	Headphones	Accessories	4	3000	2026-09-04

Contents of the CSV file (no header row):

```
101,Laptop,Electronics,2,55000,2026-09-01
102,Mouse,Accessories,5,1200,2026-09-02
103,Keyboard,Accessories,3,2500,2026-09-02
104,Monitor,Electronics,2,18000,2026-09-03
105,Headphones,Accessories,4,3000,2026-09-04
```

6. PROCEDURE

The experiment was carried out on a Linux system with Hadoop and Apache Hive 3.1.3 installed. The Hive shell was used to run HiveQL commands. Each step below shows the command used and the terminal output obtained.

Step 1 - Start Apache Hive

The `hive` command is typed in the Linux terminal to open the Hive shell.

```
$ hive
```

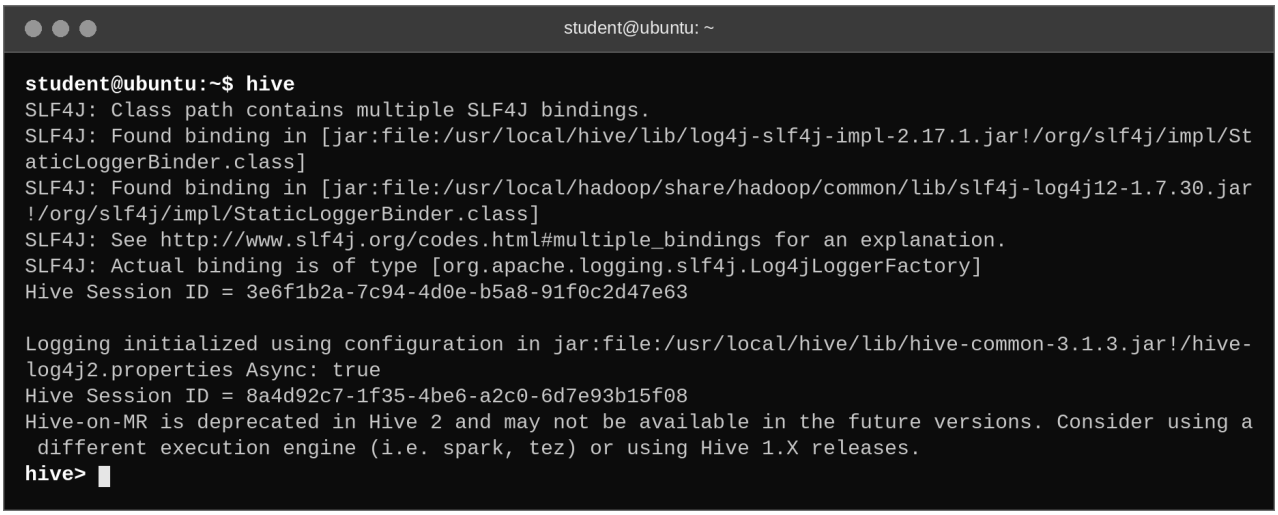


Figure 1: Starting the Apache Hive shell

The Hive shell is started to execute HiveQL commands. The prompt `hive>` indicates that the shell is ready to accept queries.

Step 2 - Create Database

A database named **retail_db** is created to store the sales table. The existing databases are listed using SHOW DATABASES and the new database is selected using USE.

```
CREATE DATABASE IF NOT EXISTS retail_db;
SHOW DATABASES;
USE retail_db;
```

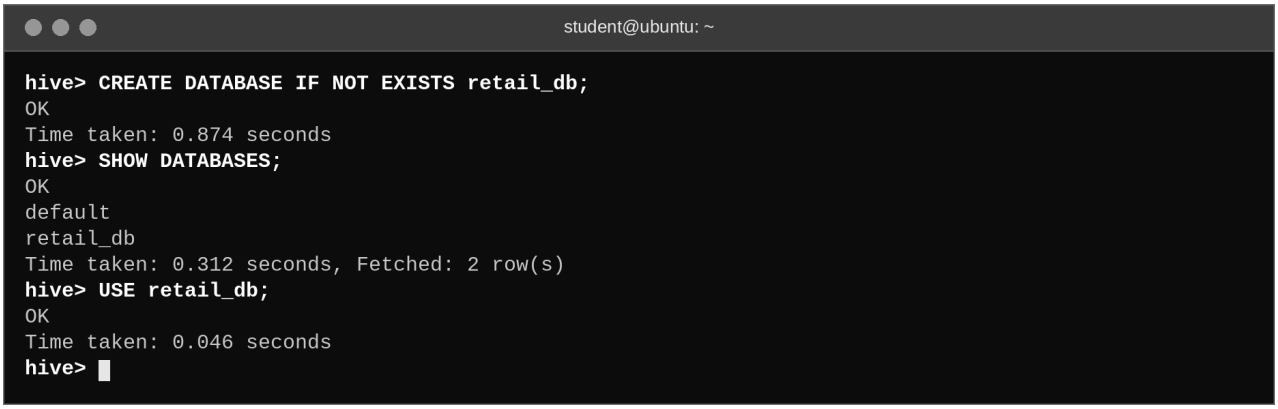


Figure 2: Creating and selecting the Hive database

The output lists the databases *default* and *retail_db*, which confirms that the database was created successfully.

Step 3 - Create Sales Table

The **sales** table is created with columns for transaction ID, product, category, quantity, price and sale date. The clause FIELDS TERMINATED BY ',' tells Hive that the columns in the input file are separated by commas.

```
CREATE TABLE sales (
transaction_id INT,
product STRING,
category STRING,
quantity INT,
price DOUBLE,
sale_date STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';

DESCRIBE sales;
```



Figure 3: Creating and describing the sales table

The DESCRIBE command displays the table structure. All six columns are created with the correct data types.

Step 4 - Load CSV Dataset

The CSV file from the local file system is loaded into the Hive table and then verified using a SELECT query.

```
LOAD DATA LOCAL INPATH '/home/student/sales.csv'
INTO TABLE sales;

SELECT * FROM sales;
```

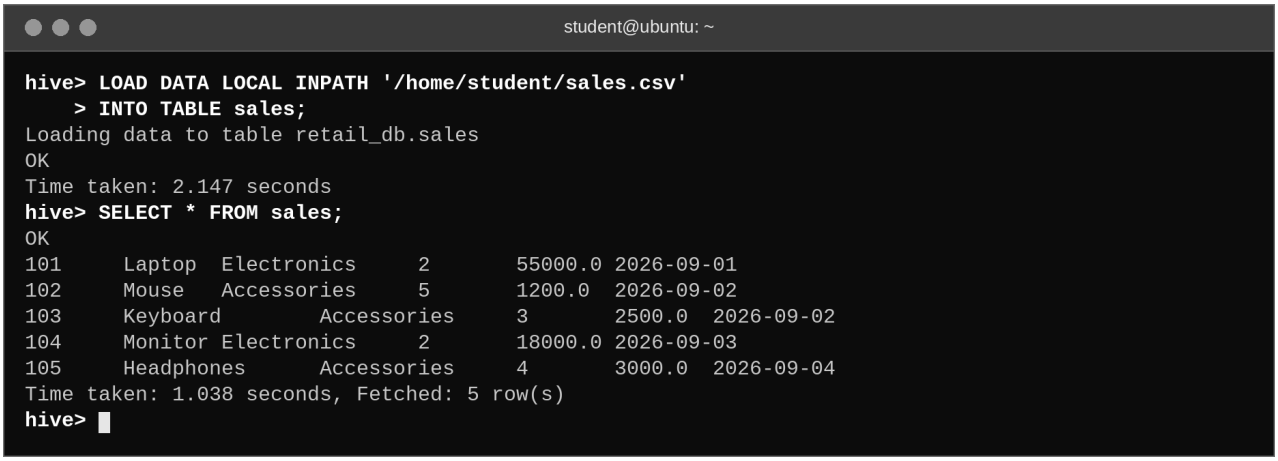


Figure 4: Loading and verifying the sales dataset

The query returns all five records, which matches the CSV file. Since the price column is of type DOUBLE, Hive displays prices with a decimal point (for example 55000.0).

Step 5 - Category-wise Quantity Analysis

The records are grouped by category and the quantity of each group is added using the SUM function.

```
SELECT category, SUM(quantity) AS total_quantity
FROM sales
GROUP BY category;
```

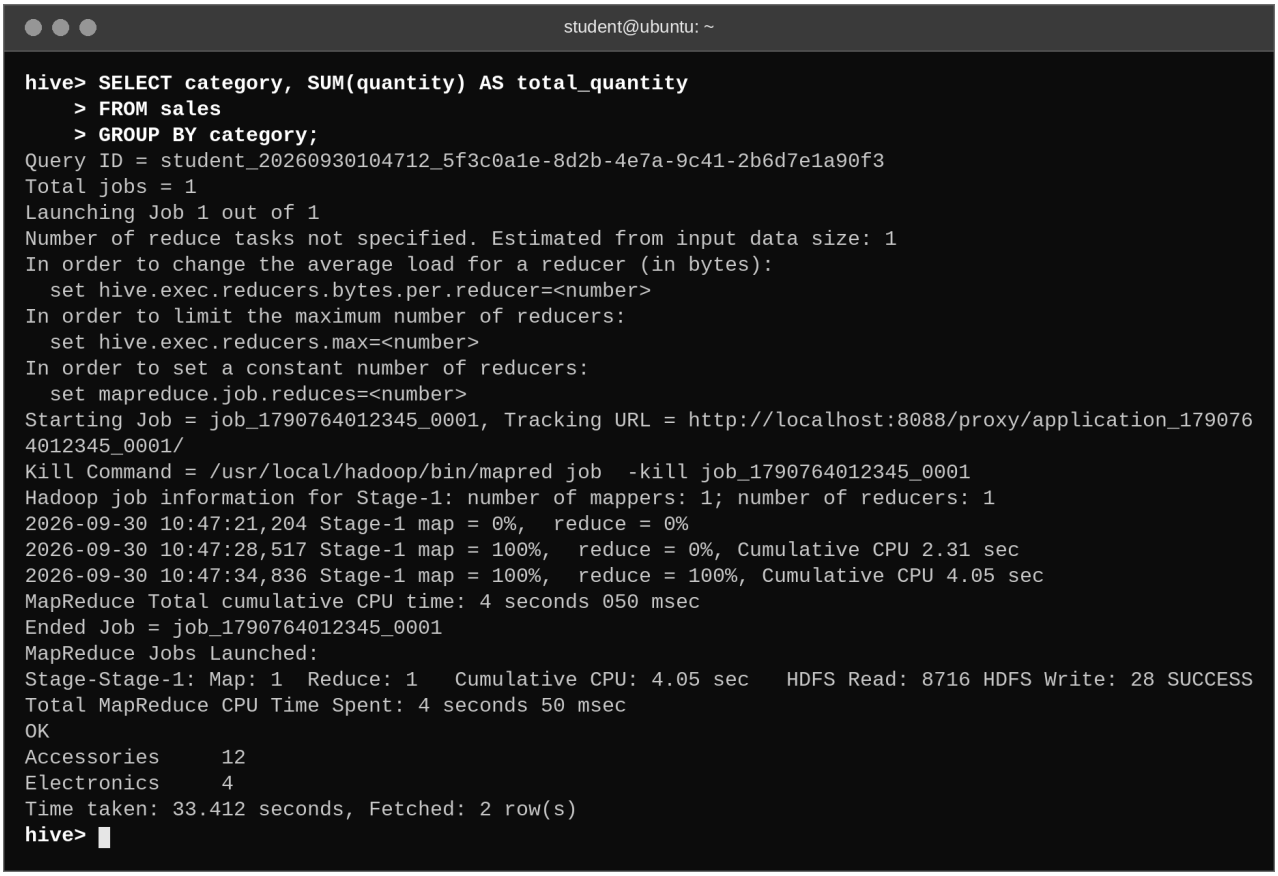


Figure 5: Category-wise quantity analysis

Hive converts the query into a MapReduce job. The result shows 12 units for Accessories (5 + 3 + 4) and 4 units for Electronics (2 + 2).

Step 6 - Category-wise Revenue Analysis

Revenue of each sale is calculated as quantity multiplied by price, and then added for each category.

```
SELECT category,
SUM(quantity * price) AS revenue
FROM sales
GROUP BY category;
```

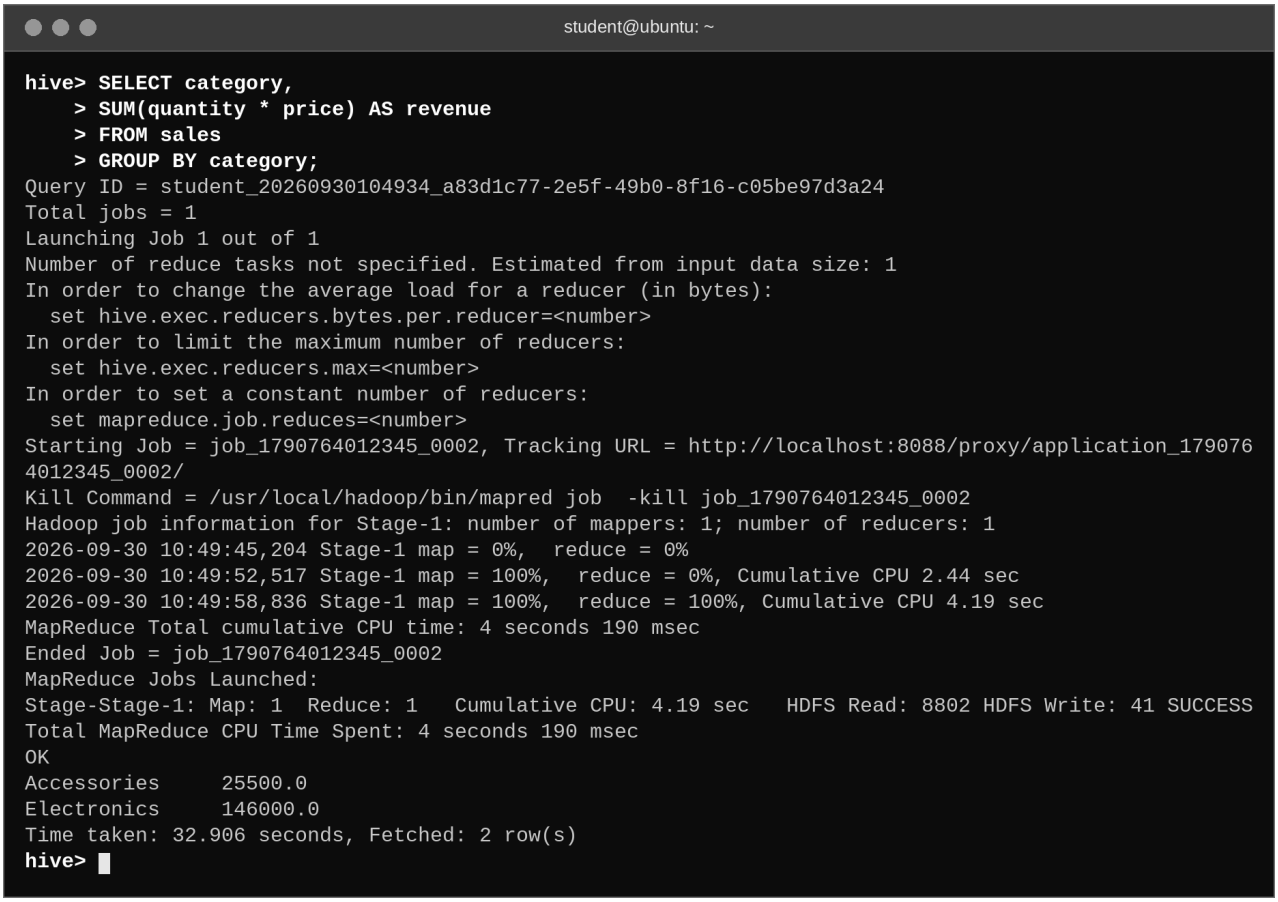


Figure 6: Category-wise revenue analysis

Manual verification of the output:

Accessories = (5 × 1200) + (3 × 2500) + (4 × 3000) = 6000 + 7500 + 12000 = **25500**
Electronics = (2 × 55000) + (2 × 18000) = 110000 + 36000 = **146000**

Step 7 - Find Top-selling Products

The products are grouped, the units sold are added, and the result is sorted in descending order. LIMIT 3 displays only the top three products.

```
SELECT product,
SUM(quantity) AS units_sold
FROM sales
GROUP BY product
ORDER BY units_sold DESC
LIMIT 3;
```

```
student@ubuntu: ~

hive> SELECT product,
    > SUM(quantity) AS units_sold
    > FROM sales
    > GROUP BY product
    > ORDER BY units_sold DESC
    > LIMIT 3;
Query ID = student_20260930105208_c4e92b60-71a3-4d58-b3f9-6e10a5d28c97
Total jobs = 2
Launching Job 1 out of 2
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1790764012345_0003, Tracking URL = http://localhost:8088/proxy/application_179076
4012345_0003/
Kill Command = /usr/local/hadoop/bin/mapred job  -kill job_1790764012345_0003
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-09-30 10:52:14,204 Stage-1 map = 0%,  reduce = 0%
2026-09-30 10:52:21,517 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 2.28 sec
2026-09-30 10:52:27,836 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 3.97 sec
MapReduce Total cumulative CPU time: 3 seconds 970 msec
Ended Job = job_1790764012345_0003
Launching Job 2 out of 2
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1790764012345_0004, Tracking URL = http://localhost:8088/proxy/application_179076
4012345_0004/
Kill Command = /usr/local/hadoop/bin/mapred job  -kill job_1790764012345_0004
Hadoop job information for Stage-2: number of mappers: 1; number of reducers: 1
2026-09-30 10:52:43,204 Stage-2 map = 0%,  reduce = 0%
2026-09-30 10:52:50,517 Stage-2 map = 100%,  reduce = 0%, Cumulative CPU 1.86 sec
2026-09-30 10:52:56,836 Stage-2 map = 100%,  reduce = 100%, Cumulative CPU 3.42 sec
MapReduce Total cumulative CPU time: 3 seconds 420 msec
Ended Job = job_1790764012345_0004
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 3.97 sec   HDFS Read: 8724 HDFS Write: 46 SUCCESS
Stage-Stage-2: Map: 1  Reduce: 1   Cumulative CPU: 3.42 sec   HDFS Read: 5210 HDFS Write: 29 SUCCESS
Total MapReduce CPU Time Spent: 7 seconds 390 msec
OK
Mouse   5
Headphones      4
Keyboard        3
Time taken: 61.275 seconds, Fetched: 3 row(s)
hive>
```

Figure 7: Top-selling products using HiveQL

Two MapReduce jobs are launched, one for grouping and one for sorting. The top-selling products are Mouse (5 units), Headphones (4 units) and Keyboard (3 units).

Step 8 - Final Analysis

A single query is used to find the number of transactions, the total units and the total revenue for each category.

```
SELECT category,
COUNT(*) AS transactions,
SUM(quantity) AS units,
SUM(quantity * price) AS revenue
FROM sales
GROUP BY category;
```

```
student@ubuntu: ~

hive> SELECT category,
    > COUNT(*) AS transactions,
    > SUM(quantity) AS units,
    > SUM(quantity * price) AS revenue
    > FROM sales
    > GROUP BY category;
Query ID = student_20260930105531_17b0d5ea-94c6-4a2d-8e73-f2c81a06b5d9
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1790764012345_0005, Tracking URL = http://localhost:8088/proxy/application_179076
4012345_0005/
Kill Command = /usr/local/hadoop/bin/mapred job  -kill job_1790764012345_0005
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-09-30 10:55:38,204 Stage-1 map = 0%,  reduce = 0%
2026-09-30 10:55:45,517 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 2.52 sec
2026-09-30 10:55:51,836 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 4.31 sec
MapReduce Total cumulative CPU time: 4 seconds 310 msec
Ended Job = job_1790764012345_0005
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 4.31 sec   HDFS Read: 8935 HDFS Write: 50 SUCCESS
Total MapReduce CPU Time Spent: 4 seconds 310 msec
OK
Accessories     3       12      25500.0
Electronics     2       4       146000.0
Time taken: 34.118 seconds, Fetched: 2 row(s)
hive>
```

Figure 8: Final sales analysis result

From the output, the overall totals are calculated as follows:

Total Transactions = 3 + 2 = **5**
Total Units Sold = 12 + 4 = **16**
Total Revenue = 25500 + 146000 = **Rs. 171500**

7. RESULT TABLE

Category	Transactions	Units Sold	Revenue
Accessories	3	12	Rs. 25,500
Electronics	2	4	Rs. 1,46,000
Total	**5**	**16**	**Rs. 1,71,500**

8. RESULT / DISCUSSION

The Apache Hive implementation successfully stored and analysed the retail sales dataset. HiveQL queries were used to group the sales records by category and calculate the total quantity and revenue.

The Accessories category contains 12 units across 3 transactions and generates Rs. 25,500 revenue. The Electronics category contains 4 units across 2 transactions and generates Rs. 1,46,000 revenue. Although Electronics has fewer units, it contributes about 85% of the total revenue of Rs. 1,71,500 because the price of each item is much higher.

Mouse, Headphones and Keyboard are the top-selling products based on units sold. The analysis demonstrates how Hive can perform aggregation, grouping, sorting and filtering operations on structured data.

9. ADVANTAGES OF APACHE HIVE

1. SQL-like HiveQL makes querying easier.
2. Suitable for large datasets.
3. Works with the Hadoop ecosystem.
4. Supports aggregation and analytical queries.
5. Can process structured and semi-structured data.
6. Reduces the complexity of writing low-level MapReduce programs.

10. APPLICATIONS

1. Retail sales analysis
2. Customer behaviour analysis
3. Log analysis
4. Data warehousing
5. Business intelligence
6. Large-scale reporting
7. Web and social media data analysis

11. CONCLUSION

Apache Hive provides an SQL-like approach for analysing large datasets stored in the Hadoop ecosystem. In this assignment, HiveQL was used to create a database and table, load a CSV dataset, perform aggregation, calculate category-wise revenue and identify the top-selling products. The results showed that Hive makes big data analysis simple, since the user can write familiar SQL-style queries instead of complex MapReduce programs.